

Hospital Management System (C++)

A console-based Hospital Management System built in C++, covering the full development lifecycle — from SRS and UML design through a working, file-persistent OOP implementation.

Built as a self-directed project to apply Programming Fundamentals and Software Engineering concepts in practice, outside of any course assignment.

Overview

The system manages four core areas of hospital record-keeping — Patients, Doctors, Appointments, and Billing — through a menu-driven console interface. Records persist across sessions using file-based storage (no database).

Features

Patient Management

- Add, update, search (by name), and view all patient records

Doctor Management

- Add, update, search (by name), and view all doctor records
- Update doctor status (active / inactive)

Appointment Management

- Book an appointment (cross-references patient and doctor records, checks doctor availability)
- Reschedule an existing appointment (date, time, or both)
- Search (by patient name) and view all appointments

Billing Management

- Generate a bill for a patient
- Search (by patient name) and view all bills

Tech Stack

Language	C++
Paradigm	Object-Oriented Programming (classes, static members, friend functions)

Data storage	Text files via <code>fstream</code> (no database)
In-memory storage	STL <code>vector</code>
IDE / Compiler	Embarcadero Dev-C++ 6.3 (TDM-GCC 9.2.0)
Documentation	Microsoft Word (SRS, UML diagrams, flowcharts)

Preview

```

===== Hospital Management System =====
1. Manage Patients
2. Manage Doctors
3. Manage Appointments
4. Manage Billing
5. Exit
Enter your choice: _

```

(Screenshots of the running program to be added.)

Project Structure

```

hospital-management-system-cpp/
├── main.cpp                                #
Full source (Patient, Doctor, Appointment,

```

```
Bill classes)
├── HMS_SRS.docx          #
Software Requirements Specification, UML
diagrams, flowcharts
└── README.md
```

This project intentionally uses a single-file architecture rather than splitting classes into separate headers, to keep the build simple in a basic IDE setup.

Getting Started

Prerequisites: a C++ compiler (TDM-GCC / MinGW or equivalent)

```
g++ main.cpp -o hms
./hms
```

On first run, the program creates its own data files (patients.txt, doctors.txt, appointments.txt, bills.txt) in the working directory.

Design Notes

- IDs are auto-generated only when a record is created, and are never reused or edited – appointments and bills reference patients/doctors by ID rather than name, since names are not guaranteed unique.

- All destructive-feeling actions (e.g. rescheduling) preserve history rather than deleting records outright.
- Every search follows the same pattern: no match → offer retry; one match → go straight to it; multiple matches → list and let the user pick.

Known Limitations

- Appointment status tracking (scheduled / completed / cancelled) is not implemented.
- Date and time are stored as plain strings; format validation is not implemented.
- Doctor search is name-based only, not by specialization.
- No double-booking prevention for appointments.
- Cancel Appointment and Update Bill were scoped during design but not implemented in this version.

See `HMS_SRS.docx` for the full requirements, UML class diagram, use case diagram, and flowcharts.

Author

Maham — BSCS Student